we mean that any given variable can at different times hold references to objects of different types.)

A Ruby *constant* is also a reference to an object. Constants are created when they are first assigned to (normally in a class or module definition). Ruby, unlike less flexible languages, lets you alter the value of a constant, although this will generate a warning message:

```
MY_CONST = 1
puts "First MY_CONST = #{MY_CONST}"

MY_CONST = 2   # generates a warning but sets MY_CONST to 2
puts "Then MY_CONST = #{MY_CONST}"
```

produces:

```
First MY_CONST = 1
Then MY_CONST = 2
```

Note that although constants should not be changed, you can alter the internal states of the objects they reference (you can freeze objects to prevent this). This is because assignment potentially *aliases* objects, creating two references to the same object.

```
MY_CONST = "Tim"
MY_CONST[0] = "J"   # alter string referenced by constant
MY_CONST  # => "Jim"
```

Scope of Constants and Variables

Constants defined within a class or module may be accessed anywhere within the class or module without needing to reference the enclosing class or module. Outside the class or module, the constant can be accessed using the *scope operator*, ::, prefixed by an expression that returns the appropriate class or module object.

Constants defined outside any class or module may be accessed unadorned or by using the scope operator with no prefix. Constants may not be defined in methods. Constants may be added to existing classes and modules from the outside by using the class or module name and the scope operator before the constant name.

```
OUTER_CONST = 99
class Const
  def get_const
    CONST
  end
  CONST = OUTER_CONST + 1
end

Const.new.get_const # => 100
Const::CONST        # => 100
::OUTER_CONST       # => 99
Const::NEW_CONST = 123
```

Global variables are available throughout a program. Every reference to a particular global name returns the same object. Referencing an uninitialized global variable returns nil.

Class variables are available throughout a class or module body. Class variables must be initialized before use. A class variable is shared among all instances of a class and its subclasses and is available within the class itself.